2026-1 지능형 로보틱스 프로젝트 발표

SAFE-CROSS V2I

비전 AI 기반 스마트 횡단보도 로봇과 자율주행 차량의 Vehicle-to-Infrastructure(V2I) 협력 제어 시스템



Team 비전
하정연(2376305), 이지현(2276246)

Contents

01

Introduction

- 프로젝트 주제
- 목표

02

Methodology

- 실행 시나리오
- Demo 시연
- 시스템 구조

03

Conclusion

- 기대효과
- 향후 계획

Motivation

1 **스쿨존** 및 **아파트 단지 내 교차로**: 매년 반복되는 어린이 등하굣길 교통사고와 인프라적 안전 공백

2 **기존 센서 중심 자율주행차의 한계**: 물리적 사각지대에 위치한 스쿨존 어린이 및 돌발 상황 대처의 불확실성







SAFE-CROSS V2I

: **비전 AI 기반 스마트 횡단보도 로봇**과 **자율주행 차량**의 Vehicle-to-Infrastructure(V2I) **협력 제어** 시스템

Solution

V2I 기반 Multi-Agent 협력 시스템 구축

- **인프라-모빌리티 연동**: 스마트 차단기 로봇과 자율주행 차량 간 실시간 데이터 교환
- **위험 사전 차단**: 사각지대 돌발 상황에 대한 선제적 안전 통제 메커니즘 구현

Goal

- **센서 한계 극복**: 차량 자체 센서의 물리적 사각지대 공백 상쇄
- **스마트 시티 실증**: 어린이 등굣길 안전을 보장하는 스마트 교통망 축소판 구현

Scenario: Safe-Cross 시스템 5단계 파이프라인

Step 1

가상 환경 및 에이전트 구축

Simulation Setup

- Gazebo Fortress 테스트베드: 도로 인프라 및 3D 에셋을 활용한 시각적 현실감 확보
- 객체 연동: 스마트 차단기 로봇, 자율주행 차량, 어린이(보행자) 모델 배치

Step 2

인프라 선제 인지

Proactive Perception

- 실시간 비전 탐지: 차단기 로봇에 탑재된 카메라와 YOLO v8 모델 연동
- 사각지대 해소: 횡단보도에 접근하는 객체(보행자, 차량)를 인프라 관점에서 선제적으로 인식

02. Methodology

Step 3

V2I 실시간 통신
V2I Communication

- 메시지 전송: 보행자 인식 즉시 ROS 2 DDS 통신망을 활용해 주변 차량에 긴급 감속 경고 메시지 브로드캐스팅

Step 4

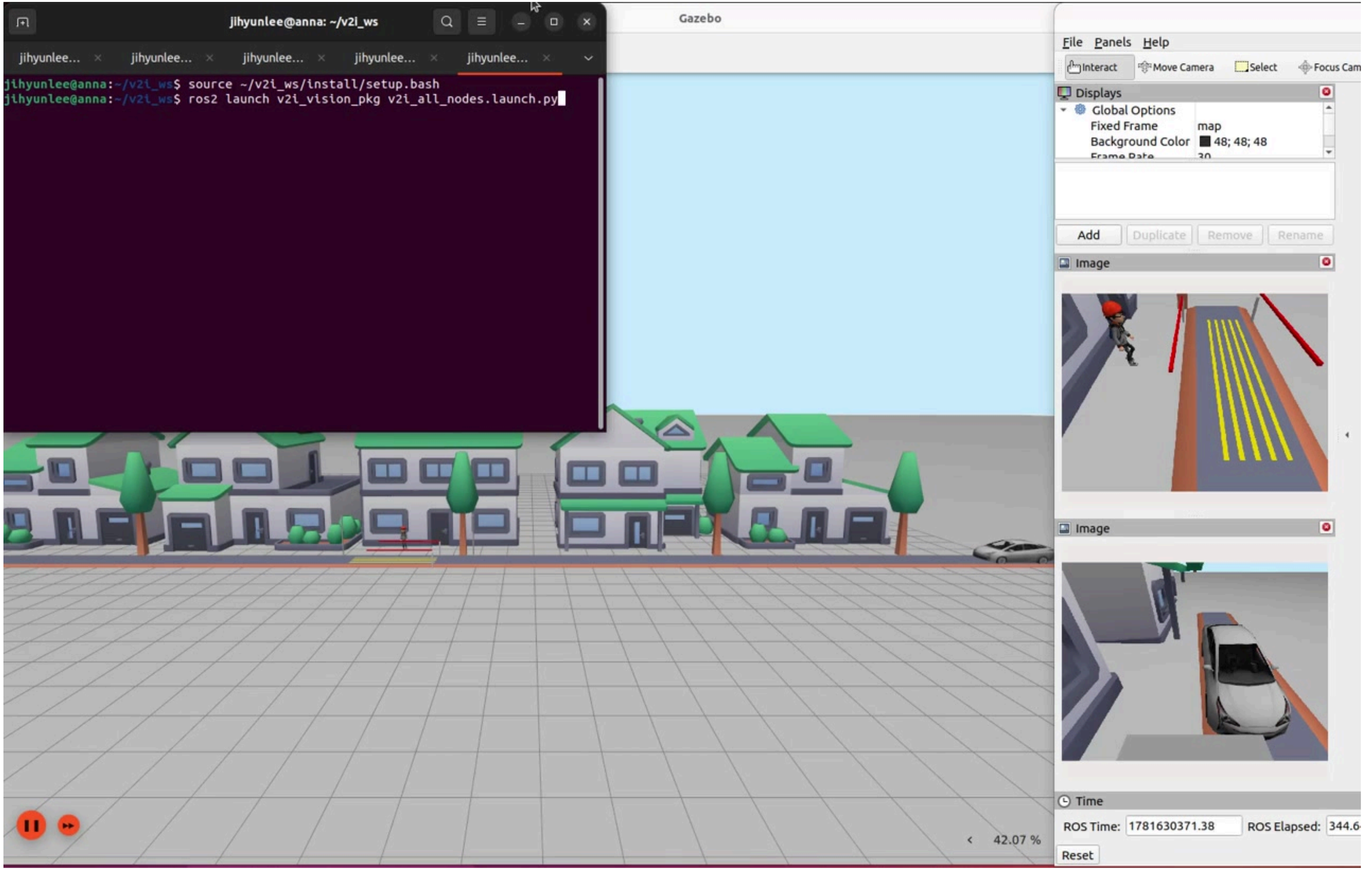
차량 협력 제어
Cooperative Control

- V2I 최우선 제어: 차량 자체 비전 센서의 식별 여부와 무관하게 인프라의 V2I 메시지를 최우선 명령으로 처리
- Soft-Stop 제어: 메시지 수신 즉시 속도를 선형적으로 감소시켜 횡단보도 정지선에 부드럽게 정차

Step 5

안전 횡단 및 흐름 재개
Safety & Resolution

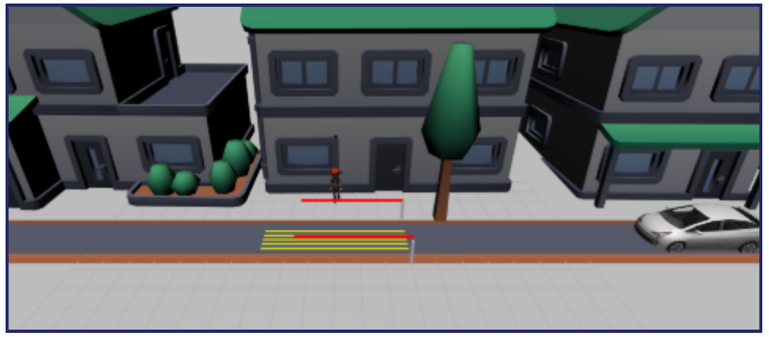
- 보행자 횡단 안내: 자율주행 차량의 완전 정차 확인 후 차단바(Joint) 개방 및 횡단 유도
- 상황 종료 및 재개: 횡단 완료 시 V2I 경고 상태 해제, 차단바 하강 및 차량 정속 주행 복귀


jihyunlee@anna: ~/v2i_ws
jihyunlee...
jihyunlee...
jihyunlee...
jihyunlee...
jihyunlee...
jihyunlee@anna:~/v2i_ws$ source ~/v2i_ws/install/setup.bash
jihyunlee@anna:~/v2i_ws$ ros2 launch v2i_vision_pkg v2i_all_nodes.launch.py
Gazebo
File
Panels
Help
Interact
Move Camera
Select
Focus Cam
Displays
Global Options
Fixed Frame
map
Background Color
48; 48; 48
Add
Duplicate
Remove
Rename
Image
Image
Time
ROS Time:
1781630371.38
ROS Elapsed:
Reset
42.07 %

1

Environment: Gazebo 기반 고정밀 교차로 테스트베드 구축

[Gazebo Fortress]



통합 시뮬레이션 월드 구축 (school_zone.sdf)

- Gazebo Fortress 기반 도심 및 등굣길 교차로 테스트베드 맵핑
- 인프라 및 로봇 모델 배치를 통한 V2I 통합 검증 환경 완성

정적 환경 에셋 (Static Assets) 적용

- Kenney City Kit 활용: 도로, 건물, 교차로 등 디테일한 도심 인프라 구현

동적 에이전트 (Dynamic Assets) 연동

- 스마트 차단기 로봇: SDF revolute joint 기반 차단바 2대 횡단보도 양끝 배치
- 자율주행 차량: Gazebo Fuel의 검증된 prius hybrid 물리/로봇 모델 적용
- 보행자(Actor): Mixamo 3D 캐릭터 모델(.dae) 및 걷기 애니메이션 추출 ➔ Gazebo Actor 시스템 완벽 연동

Pedestrian: V2I 통신 연동 보행자 제어 플러그인

[ROS 2 / C++]

독자적 C++ System Plugin 개발 (ActorControlPlugin)

- 외부 통신이 불가능한 가제보 기본 Actor 모델의 한계 극복 ➔ 시스템 플러그인(.so) 탑재 완료

ROS 2 - Gazebo 이기종 통신망 구축

- **ros_gz_bridge**를 활용한 시뮬레이터-통신 프레임워크 간 브릿지 구성
- V2I 연동: **ROS 2 DDS 통신망(/barrier_msg) Subscribe** ➔ 차단기 인프라의 보행자 제어 메시지 실시간 수신

```
this->node.Subscribe("/barrier_msg",
  &ActorControlPlugin::OnMoveCommand, this);
```

2

Pedestrian: V2I 통신 연동 보행자 제어 플러그인

[ROS 2 / C++]

상태 머신 기반 애니메이션 정밀 제어

- [대기 상태] 통제 메시지 수신 전, 제자리 모드 유지
- [횡단 상태] 이동 신호 수신 시, 가제보 엔진의 Update 사이클에 직접 개입
  ➔ 걷기 모션 시간(dt) 누적 및 World Pose 좌표를 강제 갱신하여 자연스러운 Walk 동작 구현

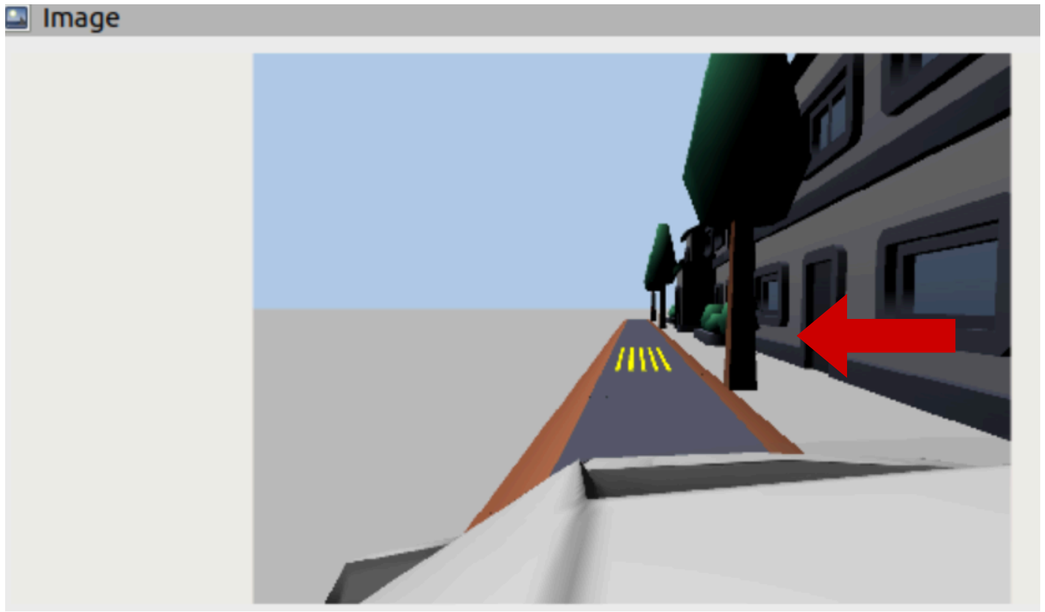
```
// === 1. 애니메이션: shouldMove일 때만 시간 누적 (= 걷기 모션 재생) ===
//    shouldMove가 false면 AnimationTime을 그대로 두므로
//    actor는 첫 프레임 자세(차렷 자세)로 정지
if (this->shouldMove)
{
  auto animTimeComp =
      _ecm.Component<gazebo::components::AnimationTime>(this->actorEntity);
  if (animTimeComp)
  {
    auto newTime = animTimeComp->Data() + _info.dt;
    *animTimeComp = gazebo::components::AnimationTime(newTime);
    _ecm.SetChanged(this->actorEntity,
      gazebo::components::AnimationTime::typeId,
      gazebo::ComponentState::PeriodicChange);
  }

  // === 2. 이동 신호 시 X 오프셋 누적 (월드 X축 방향으로 전진) ===
  this->offset_x -= 0.1;
}
```

3

Vehicle: 인프라 협력 주행 및 Soft-Stop 제어 노드

[ROS 2 / Python]



차량 시각 센서 통합 및 실시간 모니터링

- 자율주행 차량 전면 범퍼에 카메라 센서 장착
- RViz2를 활용한 1인칭 전방 시야 실시간 스트리밍 검증

✓ 자체 센서 기반 인지의 물리적 한계 실증

- 가로수 등 지형지물에 의해 발생하는 '물리적 사각지대' 직접 확인
- 차량 단독 비전 센서로는 가려진 보행자를 인식하기 어려움

→ 외부 인프라(스마트 차단기)의 V2I 개입 당위성 확보

3

Vehicle: 인프라 협력 주행 및 Soft-Stop 제어 노드

[ROS 2 / Python]

ROS 2 기반 V2I 양방향 통신망 제어

- 수신: 차단기 로봇의 보행자 감지 신호를 최우선 제어 명령으로 처리
- 송신: 차량 완전 정차 시 차단기 로봇에게 역으로 상태를 보고하여 안전 확인

선형 가감속 프로파일(Soft Stop) 및 상태 머신(FSM) 연동

- **정속 주행**: 10Hz 주기의 상시 제어 루프를 통해 안정적인 속도 명령 발행
- **선형 감속**: 통제 신호 수신 시, 급제동을 방지하고 제동 안전성을 확보하기 위해 선형 감속 제어 적용
  ➔ 정지선 앞 Soft Stop 수행
- **정차 후 재출발**: 해제 신호 수신 시, 목표 속도까지 부드러운 가속 프로파일을 따라 주행을 재개하는 유기적 상태 머신 구성

1 비동기적 상태 제어

- 역할: 인프라(차단기)로부터 /v2i_alert 신호를 수신하여 차량의 주행 상태(is_stopping)를 결정하는 뇌
- 동작 원리:
  - [위험 감지 시] True 수신 시 is_stopping = True로 전환하여 즉각적인 감속 모드 진입
  - [상황 해제 시] False 수신 시 목표 속도를 복구하고, 차단기 측으로 '주행 재개 신호'를 송신하여 양방향 상태 동기화 완료

```python
def alert_callback(self, msg):
    """차단기 로봇의 신호 수신"""
    if msg.data == True and not self.is_stopping:
        self.get_logger().info("🔴 [안전 통제] v2i 정지 메세지 수신. Soft Stop을 시작합니다.")
        self.is_stopping = True

    elif msg.data == False and self.is_stopping:
        self.get_logger().info("🟢 [통제 해제] 어린이(보행자) 횡단 완료. 주행을 재개합니다.")
        self.is_stopping = False
        # 다시 목표 속도로 엑셀을 밟기 위한 초기화
        self.target_speed = 15.0
        self.stopped_published = False  # 다음 정지 시 다시 신호 보낼 수 있도록 초기화

        # 차량이 다시 움직임 → barricade_control_node에 알림
        stopped_msg = Bool()
        stopped_msg.data = False
        self.stopped_pub.publish(stopped_msg)
```

2 실시간 동역학 제어 (Time-Driven)

- 역할: 10Hz(0.1초) 주기로 작동하며, 상태 변수에 따라 물리적인 모터 속도를 계산하고 제어하는 근육
- 동작 원리:
  - [선형 감속] 감속 모드 시, 매 루프마다 설정된 변화량(decel_step)만큼 속도를 빼앗아 급제동을 방지하는 Soft-Stop 구현
  - [정차 보고] 속도가 0.0에 도달하는 즉시, 통신 과부하 방지를 위해 1회만 정지 신호(stopped_published)를 인프라로 퍼블리시
  - [좌표 매핑] 계산된 최종 속도를 3D 차량 모델의 로컬 Y축 벡터로 변환하여 가제보 엔진에 전달

```
if self.is_stopping:
    # 브레이크 밟기 (속도를 서서히 0으로)
    if self.current_speed > 0.0:
        self.current_speed -= self.decel_step
    if self.current_speed < 0.0:
        self.current_speed = 0.0

    # 완전 정지 도달 시 barricade_control_node로 신호 1회 발행
    if self.current_speed <= 0.0 and not self.stopped_published:
        self.get_logger().info("🔴 차량 완전 정지. 차단기 노드에 정지 신호 전송.")
        stopped_msg = Bool()
        stopped_msg.data = True
        self.stopped_pub.publish(stopped_msg)
        self.stopped_published = True
else:
    # 엑셀 밟기 (출발 신호를 받으면 서서히 목표 속도까지 가속)
    if self.current_speed < self.target_speed:
        self.current_speed += 0.5 # 0.1초당 가속
```

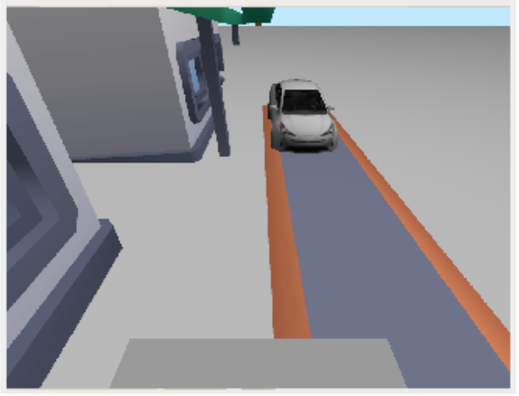
4

Integration: 차단기 하드웨어 연동 및 Safe-Cross 통합 검증

[YOLOv8 / ROS 2 / Python]



횡단보도



도로

YOLOv8 이중 카메라 감지 노드

- 모델: yolov8n.pt (경량 모델, 실시간 추론)
- 카메라 입력 주기: 30Hz
- 역할:
  - **횡단보도 카메라(/camera/image_raw):** 보행자(class 0) 감지
  - **도로 카메라(/camera/road_raw):** 차량(car/truck/bus/motorcycle) 감지

  ➔ 3프레임 연속 확인 후 안정적 감지 판정
- 동작:
  - 차단기 노드에 보행자, 차량 감지 여부 메세지 상시 발행
  - 보행자 + 차량 동시 감지 → 차량 제동 신호 발행

```
def _check_and_alert(self, confirmed_absent):
    if self.stable_pedestrian and self.stable_car and not self.alert_sent:
        self.get_logger().info('보행자 + 차량 감지! 차량 정지 신호 전송.')
        alert = Bool()
        alert.data = True
        self.v2i_alert_pub.publish(alert)
        self.alert_sent = True
```

4

Integration: 차단기 하드웨어 연동 및 Safe-Cross 통합 검증

[YOLOv8 / ROS 2 / Python]

차단기 제어 노드

- 수신: YOLO 카메라의 보행자, 차량 감지 메세지 및 차량의 완전 정지 상태 수신
- 송신:
  - [차량 완전 정지 확인] 차단바 개방 + 보행자 출발 신호 발행
  - [보행자 감지 + 차량 미감지] 3초 대기 → 차단바 개방 + 보행자 출발 신호 발행
  - [보행자 횡단 완료] 차단바 복귀 + 차량 재출발 신호 발행

TTS 음성 안내

- 차단바 개방 시 “건너가세요” 음성 안내
- 노드 시작 시 미리 생성해 재생 지연 없음 (gTTS + pygame)

```python
def check_and_open(self):
    """차단바 올릴 조건 확인"""
    if not self.pedestrian_present or self.barricade_raised:
        if not self.pedestrian_present:
            self.no_vehicle_timer_start = None
        return

    # 차량 있고 정지 완료 → 즉시 열기
    if self.vehicle_present and self.vehicle_stopped:
        self._open_barricade()

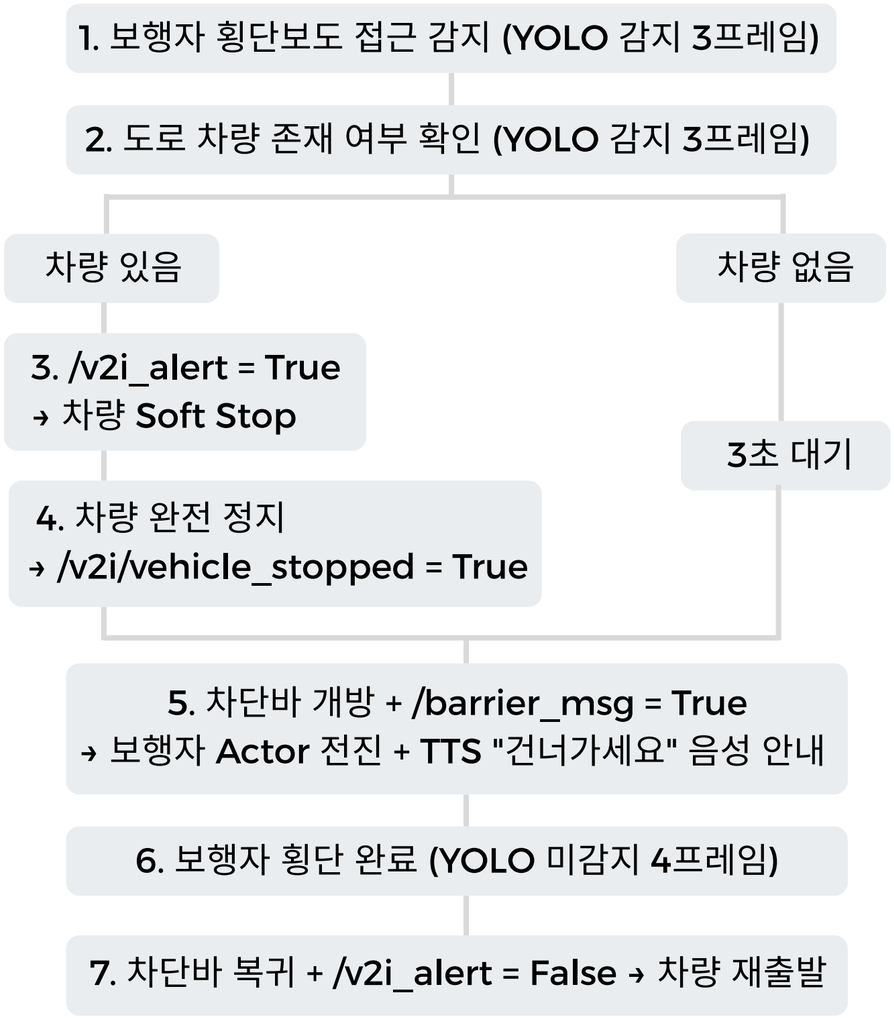
    # 차량 없음 → 3초 대기 후 열기
    elif not self.vehicle_present:
        if self.no_vehicle_timer_start is None:
            self.no_vehicle_timer_start = self.get_clock().now()
            self.get_logger().info('차량 없음 감지. 3초 후 차단바 개방 예정.')
        else:
            elapsed = (self.get_clock().now() - self.no_vehicle_timer_start).nanoseconds / 1e9
            if elapsed >= 3.0:
                self.get_logger().info('차량 없음 3초 경과. 차단바 개방.')
                self._open_barricade()
```

4

Integration: 차단기 하드웨어 연동 및 Safe-Cross 통합 검증

[YOLOv8 / ROS 2 / Python]

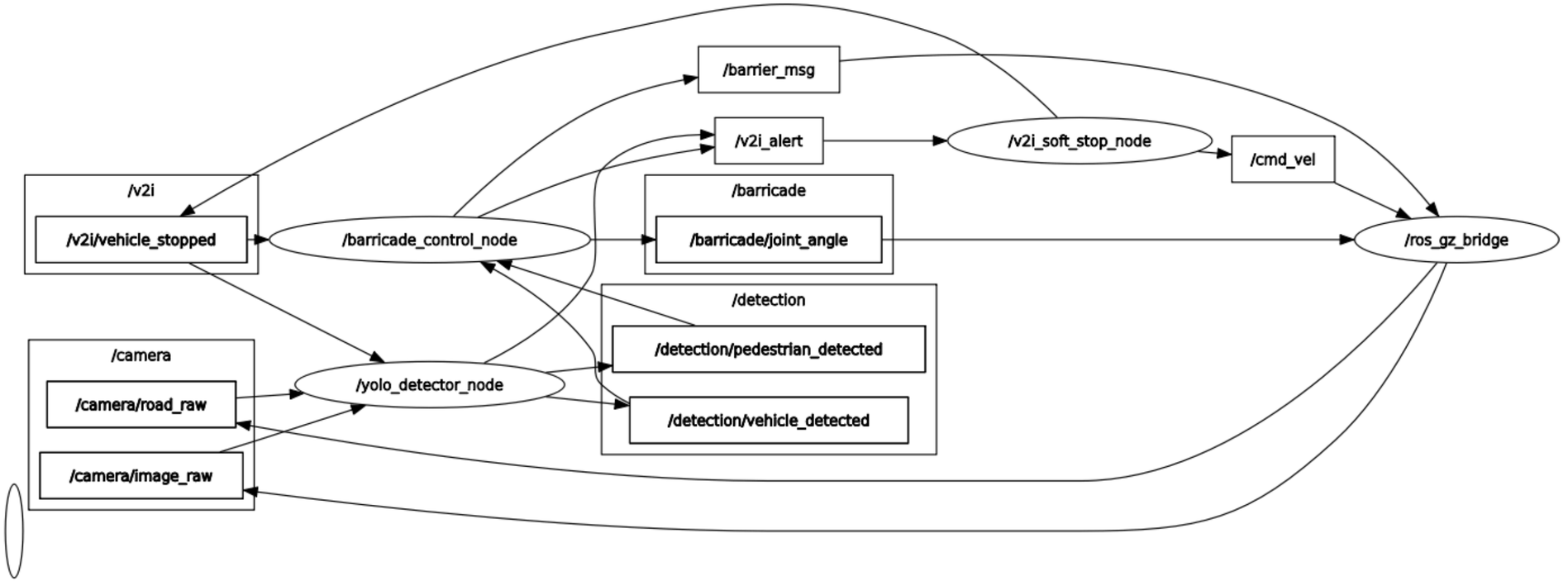
전체 Safe-Cross 시나리오 통합 검증 흐름



4

Integration: 차단기 하드웨어 연동 및 Safe-Cross 통합 검증

[YOLOv8 / ROS 2 / Python]



4

Integration: 차단기 하드웨어 연동 및 Safe-Cross 통합 검증

[YOLOv8 / ROS 2 / Python]

```python
def generate_launch_description():
    return LaunchDescription([
        Node(
            package='ros_gz_bridge',
            executable='parameter_bridge',
            name='ros_gz_bridge',
            arguments=[
                '/cmd_vel@geometry_msgs/msg/Twist]ignition.msgs.Twist',
                '/camera/image_raw@sensor_msgs/msg/Image[ignition.msgs.Image',
                '/camera/road_raw@sensor_msgs/msg/Image[ignition.msgs.Image',
                '/camera/driver_raw@sensor_msgs/msg/Image[ignition.msgs.Image',
                '/barricade/joint_angle@std_msgs/msg/Float64]ignition.msgs.Double',
                '/barrier_msg@std_msgs/msg/Bool]ignition.msgs.Boolean',
            ],
            output='screen',
        ),
        Node(
            package='v2i_vision_pkg',
            executable='yolo_detector_node',
            name='yolo_detector_node',
            output='screen',
        ),
        Node(
            package='v2i_vision_pkg',
            executable='barricade_control_node',
            name='barricade_control_node',
            output='screen',
        ),
        Node(
            package='v2i_vehicle_pkg',
            executable='v2i_soft_stop_node',
            name='v2i_soft_stop_node',
            output='screen',
        ),
```

노드 구성

- ROS2 노드 3개 + Gazebo 플러그인이 토픽으로 상태를 공유하며 협력
  - yolo_detector_node: YOLOv8로 보행자/차량 감지
  - barricade_control_node: 차단바 개폐 + TTS 음성 안내
  - v2i_soft_stop_node: 차량 속도 제어 (감속/재출발)
  - ActorControlPlugin: 보행자 Actor 이동 제어 (C++ Gazebo 플러그인)

통합 실행

- launch.py: ROS2 노드 3개 실행 + ros_gz_bridge 실행
- ActorControlPlugin은 Gazebo 실행 시 SDF에서 자동 로드

Safe-Cross V2I 기대효과

🚶 물리적 사각지대 해소를 통한 보행 안전 극대화

✓ 차량 센서 단독 인지의 한계를 V2I 인프라(차단기 로봇) 협력으로 보완
✓ 스쿨존 및 아파트 단지 내 어린이 돌발 사고 예방 및 교통 안전망 구축

🚗자율주행 시스템의 제어 신뢰도 및 승차감 향상

✓ Soft-Stop 알고리즘을 통한 안정적인 감속 프로파일 확보 및 급제동 사고 방지
✓ 인프라-모빌리티 간 양방향 통신 데이터 기반의 정밀한 상태 동기화 실현

🚥미래형 스마트 시티 교통 패러다임 제시

✓ 지능형 인프라가 교통 흐름에 직접 개입하는 Multi-Agent 협력 모델의 가능성

Future Work

V2I 보행자 안전 시스템의 다음 단계

AI 비전 알고리즘 고도화 및 엣지 컴퓨팅 최적화

- 다양한 기상 조건 및 야간 환경에서의 YOLO v8 탐지 정확도 향상
- 실시간성 확보를 위한 가속화 및 Latency 최소화

Multi-Vehicle 환경 확장 및 군집 주행 연동

- 단일 차량을 넘어 다수의 자율주행 에이전트가 포함된 복잡한 교차로 시나리오 확장
- 인프라 신호에 따른 차량 간 군집 주행 및 최적 교통 흐름 제어 알고리즘 연구

Safe-Cross V2I의 발표 마칩니다.

THANK YOU



Team 비전
하정연(2376305), 이지현(2276246)